Conversational-Search Token-Cost Optimization: A Savings Forecast and the Case for TIP as a Cost Platform

Sean Poyner
Use Case Engineering, Marriott D&AI

June 2026 (DRAFT - assumptions flagged for calibration)

Abstract

Marriott's guest-facing AI conversational search (the `mi-conversational-ai-agent` / "Ask Bonvoy" NLS experience) runs on **Gemini 3.1 Flash-Lite via the TIP.ai LiteLLM proxy** (Jira GENAI-1412, WEB-194505) and is under active cost review (the Cost Optimization Alignment workstream). This report forecasts how far its token cost can be cut *at fixed answer quality*. Under an explicit, parameterized model (calibrate against Adobe event206-208 and Dynatrace token telemetry), the monthly run-rate baseline is on the order of **$540K/month** at current assumptions, of which an **expected ∼75%** is recoverable. The three levers, in order of impact: (1) **caching** the large fixed prefix re-sent every turn (∼35% alone, 22–42% across a hit-rate sweep, zero quality risk); (2) **context/RAG pruning** + history compaction (∼57% combined); and (3) a **default model swap to `gemma4:31b` on Vertex** - a *managed, in-GCP* model that is ∼42% cheaper than Flash-Lite at this token mix and, in our head-to-head measurement, of *statistically on-par* quality (∼75% combined, up to 87% aggressive). The incumbent Flash-Lite is *not* Google's cheapest competent tier, so model selection is a real lever here; by contrast, *self-hosting* the same model on Amazon is ∼26× more expensive per token and is a data-residency/control play, not a cost lever. We recommend a prescriptive sequence (caching → pruning → `gemma4:31b`-on-Vertex default), gated by a strict do-no-harm quality eval, delivered as a reusable **TIP SDK platform capability** (router + caching + context-pruning middleware) emitting spend metrics to the control plane.

Executive summary

- **Confirmed setup:** Gemini 3.1 Flash-Lite ($0.25 in / $1.50 out per 1M) via the TIP.ai LiteLLM proxy (GENAI-1412, WEB-194505). Optimization middleware lands exactly where traffic already flows.
- **Baseline (assumption-driven):** ∼24M conversations/month → ∼1.9T input tokens/month → ∼**$540K/month** (∼$0.022/conversation). Input is ∼98% of tokens and scales *quadratically* with conversation length, because the 15k+ system prompt and RAG are re-sent every turn and history grows.
- **Lever 1 - caching (no quality risk):** 35% (expected); 22–42% across a 50–95% hit-rate sweep.
- **Lever 2 - context/RAG pruning:** → ∼57% combined.
- **Lever 3 - swap default to `gemma4:31b` on Vertex:** managed, in-GCP, ∼42% cheaper than Flash-Lite at *measured on-par* quality → **∼75% combined (expected), up to 87% aggressive**.

- **Self-host on Amazon is NOT a cost lever:** the same model self-hosted is ~$4/1M (owned-compute est) vs `gemma4:31b`-on-Vertex's ~$0.16/1M; justify it on data residency/control, not unit cost.
- **Recommendation:** caching first, then pruning, then the `gemma4:31b`-on-Vertex default behind a strict quality gate; build it once as a TIP SDK platform capability feeding the control plane.

Three audiences: a Gen-AI exec cost view (§3), a search-team lever playbook (§4, §5), and a TIP platform case (§7).

1 Context and scope

Guest-facing conversational search (DDC events 206-208; the AI-for-Search workstream; products `mi-conversational-ai-agent`/"Ask Bonvoy"/NLS/Polaris) runs on GCP and, per Jira, on **Gemini 3.1 Flash-Lite through the TIP.ai LiteLLM proxy**. Token costs are capitalizable and under exec attention (Paul Dyrwal). Sean has built a LiteLLM model router for the TIP SDK (delivery imminent), and Kenny Hammerlund's control plane tracks per-agent spend with conversational search as its first use case. Objective (from alignment): a *max-defensible savings curve* at fixed quality, with the do-no-harm constraint (no regression to Step-Conversion-to-RLM or Visitor-Conversion) treated as sacred.

2 Method

A single parameterized model (`scripts/forecast/conv_search_forecast.py`); every input is explicit and auditable, to be calibrated against Adobe event206-208 volume, Dynatrace token telemetry, and the Egor/Kenny cost baseline. Model price/quality options reuse the LiteForge cross-provider cost-quality study (25 models, graded benchmarks, bootstrap CIs). Rigor: conservative/expected/aggressive **bands**, a **sensitivity** (tornado) analysis, and bootstrap CIs where inputs are measured (model quality). Horizon: monthly run-rate.

Why input scales quadratically. For a conversation of T turns with fixed system S, RAG R, per-turn query q and output o, total input tokens are

$$\text{input} = T\,(S + R + q) + (q + o)\,\frac{T(T-1)}{2},$$

because S and R are re-sent every turn and history accumulates. The first term is what caching attacks; the second is what history compaction attacks.

3 Baseline forecast

At current assumptions (270M Bonvoy members × 3% adoption × 3 conversations/mo × 4 turns; system 15k, RAG 4k, output 400 tokens; Gemini 3.1 Flash-Lite at $0.25/$1.50 per 1M): ~24.3M conversations/month, ~1.91T input + 38.9B output tokens/month, **~$540K/month**, $0.022/conversation. The baseline is most sensitive to conversations-per-user, adoption, and input price (Figure 4); the 3% adoption figure scales the whole result linearly and should be validated against Adobe first.

4 The savings levers

Figure 1 shows cumulative savings as the levers are applied in order.

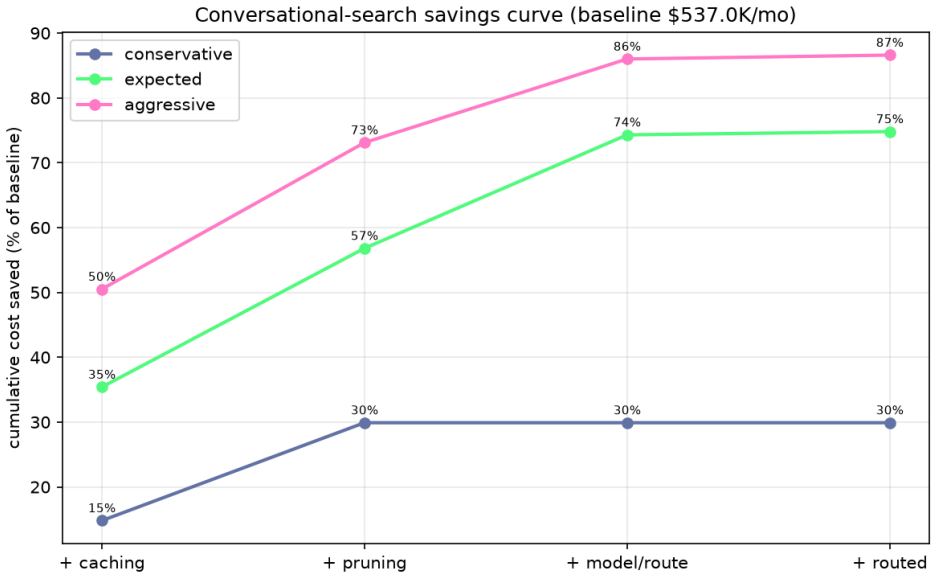

Figure 1: Cumulative cost saved vs baseline as levers stack, for three scenarios. Caching and pruning carry the first ∼57%; the `gemma4:31b`-on-Vertex default swap adds ∼18 more points (expected).

(1) Caching - the safest win. The 15k+ system prompt plus the stable share of RAG is a fixed prefix re-sent every turn; caching it (Vertex context caching) bills those tokens at a fraction of input price on a hit. Figure 2: **22% saved at a 50% hit rate, rising to 42% at 95%.** Ship this first - no model change, no quality risk. (Open item: confirm Vertex/TIP caching support and the cached-token discount.)

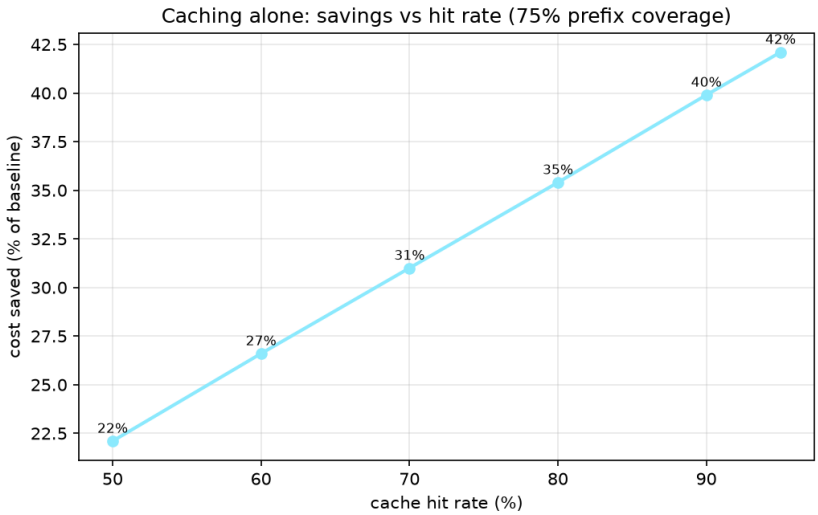

Figure 2: Caching alone: cost saved vs cache hit rate at 75% prefix coverage.

(2) Context + RAG pruning, history compaction. Trimming the system prompt, tightening retrieval, and compacting/windowing history attack the variable cost (and the quadratic history term). With caching this reaches ∼57% (expected).

(3) Swap the default to `gemma4:31b` on Vertex. The incumbent Gemini 3.1 Flash-Lite ($0.25/$1.50) is *not* Google's cheapest competent tier. Managed, per-token **`gemma4:31b` on Vertex** (published $0.15/$0.60) is ∼42% cheaper at this input-heavy mix (Figure 3). Crucially, this is not a quality trade: in a head-to-head on the same 290 graded prompts, `gemma4:31b` scored **0.866 vs Flash-Lite's 0.859** (paired difference +0.007, 95% CI $[-0.017, +0.031]$) - *statistically on par*, tied per-benchmark with a slight edge on GSM8K and MBPP. Because quality is non-inferior, this is a **default swap for ∼all traffic**, not just an easy-tail route, taking expected savings from ∼57% to ∼75%. (A hard tail can still escalate to Flash-Lite/Gemini Pro for safety, via an escalate-on-failure cascade gated by the grounding check rather than an up-front difficulty predictor; see Section 8.) A learned router adds little over this static swap at near-parity (the LiteForge finding), so we credit selection, not routing cleverness.

(4) Self-host on Amazon - residency, not cost. The same model self-hosted on Amazon (Bedrock/EKS, owned-compute estimate) is ∼$4/1M - roughly ∼26× the managed `gemma4:31b`-on-Vertex price and ∼15× Flash-Lite (Figure 3). Self-host only approaches parity at very high utilization; its value here is data residency/control or custom fine-tuning, argued on those terms.

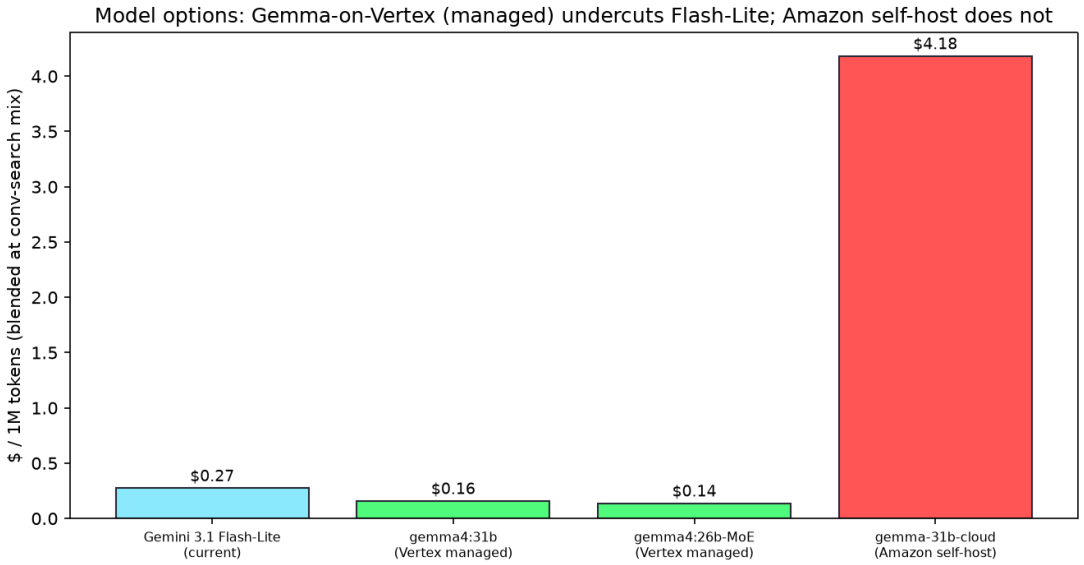

Figure 3: Blended $/1M at the conversational-search token mix. Managed `gemma4:31b`-on-Vertex undercuts the incumbent Flash-Lite at on-par quality; Amazon self-host does not (residency option).

5 Quality gate (do-no-harm)

No change ships unless it is *strictly non-inferior* to the Gemini-3.1-Flash-Lite baseline. The `gemma4:31b` swap already clears this on public benchmarks (paired diff +0.007, CI $[-0.017, +0.031]$ above); the production gate repeats the same test on **real conversational-search queries** using the LiteForge evaluation harness with Lakshmi Nimmagadda's existing eval set, scoring each candidate by an LLM judge *and* a grounding/faithfulness check (guest answers must be supported by retrieved inventory/policy - no hallucinated rates), with bootstrap CIs. Recommended rollout: shadow → gated A/B with auto-rollback on KPI regression; execution detail left to the search team.

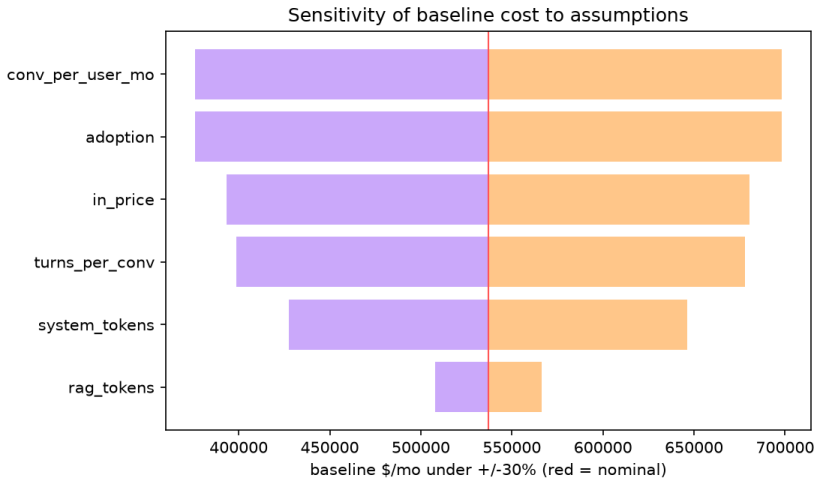

Figure 4: Sensitivity of the monthly baseline to each assumption (±30%). Volume (conversations/user, adoption) and price dominate; validate these first.

6 Recommendation

Primary sequence (prescriptive):

1. **Enable prefix caching** (system prompt + static RAG). Biggest safe win (22–42% alone). Confirm Vertex/TIP caching support first.
2. **Prune context** (trim system prompt, tighten retrieval, compact history) → ∼57% combined.
3. **Swap the default to `gemma4:31b`-on-Vertex** (managed, in-GCP) behind the quality gate → ∼75% combined.

Options on the curve: the conservative/expected/aggressive band (∼30% / ∼75% / ∼87%) lets leadership pick an operating point against effort and risk. **Self-host on Amazon** only if data-residency/control or fine-tuning requires it - not a unit-cost win. (Capitalization note: dev-phase token costs are capitalizable; finance to confirm treatment.)

7 TIP as a cost-optimization platform

These levers should be built once, in the TIP SDK, as a reusable platform capability: the already-announced LiteLLM **router** plus **caching** and **context-pruning** middleware, emitting **spend metrics** to Kenny's control plane (which both visualizes spend and governs the routing/guardrail config). Conversational search already calls the TIP LiteLLM proxy (WEB-194505), so it is the first and highest-volume tenant; the same capability then serves ConEng consumers and other agents. This analysis forecasts the savings; the router policy itself is already designed. One refinement from the routing studies below: the hard-tail mechanism should be an *escalate-on-failure cascade*, not an up-front difficulty predictor.

8 Routing the hard tail: evidence from routing studies

Lever 3 makes `gemma4:31b` the default; a small fraction of hard queries may still need a stronger model. How that escalation is built matters, and two companion studies (`docs/paper/liteforge-routing.tex` and `docs/agentic-routing-study.md`) make the design concrete.

The `gemma4:31b` default is now triply corroborated. Three independent measurements pick the same model: this study's head-to-head (0.866 vs Flash-Lite 0.859), the cross-provider cost-quality frontier (0.865, 96% of Sonnet, the most parameter-efficient strong open-weight model), and an agentic tau-bench study where `gemma4:31b` as the cheap tier *outscored* a 397B model at a fraction of the cost. The lever-3 swap rests on convergent evidence, not a single benchmark.

Predict-difficulty-up-front is the wrong hard-tail mechanism for hard cases. A learned difficulty router only recovers cost when difficulty is legible from the input. It works on QA-style prompts (RouterBench, APGR 0.31 for the embedding head), which conversational search resembles, so a lightweight router *is* viable here for the easy/hard split. But it collapses to random on agentic, multi-turn tasks where difficulty emerges only during execution. The robust, domain-independent mechanism is a **cascade**: serve `gemma4:31b`, and escalate to Flash-Lite or Gemini Pro only when the cheap answer fails a check.

Verification is cheaper here than in agentic settings. The agentic study found that escalation triggers vary widely: give-up signals are useless, self-consistency (sample the cheap model a few times, escalate on disagreement) recovers most of the headroom for free, and an LLM-judge helps only with a strong, calibrated verifier (silent agentic failures are nearly unverifiable). Conversational search is easier to verify than an agentic booking: the do-no-harm gate already specifies a grounding/faithfulness check (Section 5), which doubles as the escalation trigger. So a conv-search cascade can gate on grounding-failure or low-confidence and escalate that tail to Flash-Lite/Pro, capping the quality regression while keeping the bulk of the ∼75% savings. Self-consistency is the free fallback trigger; an LLM-judge is justified only if a strong, cheap verifier is available.

9 Caveats and calibration

The numbers are a transparent model, not measured spend. Before external use, calibrate: (i) adoption and conversations/turns per user from Adobe event206-208; (ii) token mix from Dynatrace/TIP token telemetry; (iii) the in-tenant Flash-Lite and `gemma4:31b`-on-Vertex prices and which caching the Vertex/TIP path supports; (iv) reconcile the baseline against the Egor/Kenny Cost Optimization Alignment numbers; (v) re-run the quality gate on real search queries (the +0.007 on-par result is on public benchmarks). The model (`conv_search_forecast.py`) regenerates every figure and `forecast.json` once inputs are updated.